Kubernetes Liveness Probe

A **Liveness Probe** in Kubernetes is a mechanism used to determine whether a **running container is still healthy and capable of functioning**.

In simple terms:

Liveness Probe answers: "Is this container alive, or has it become stuck/broken and needs to be restarted?"

If the liveness probe **fails repeatedly**, Kubernetes considers the container unhealthy and **restarts the container**.

1. Why do we need a Liveness Probe?

A container can be:

- Running
- Process exists
- Container status is Running

...but the application inside it may no longer be functioning correctly.

For example:

Pod

└── Container

└── Java Application

├── Process → Running

├── JVM → Running

└── Application → Hung ❌

Kubernetes normally checks whether the container process is running.

If PID 1 is still running, Kubernetes may think:

Container → Running → Healthy

But the application could actually be:

Deadlock

Infinite loop

Thread exhaustion

Memory/resource problem

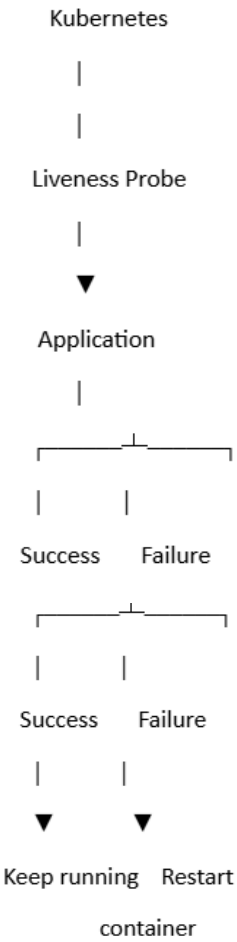
Internal application failure

Stuck connection pool

Application no longer responding

2. What does Liveness Probe actually do?

The basic flow is:



If the application responds successfully:

Probe → SUCCESS

Kubernetes leaves the container running.

If the application repeatedly fails:

Probe → FAILURE

Probe → FAILURE

Probe → FAILURE

↓

Container restart

---

3. Important distinction: Liveness ≠ Application Availability

This is one of the most important concepts.

A liveness probe does **not necessarily mean**:

"Can users access my application?"

Instead, it means:

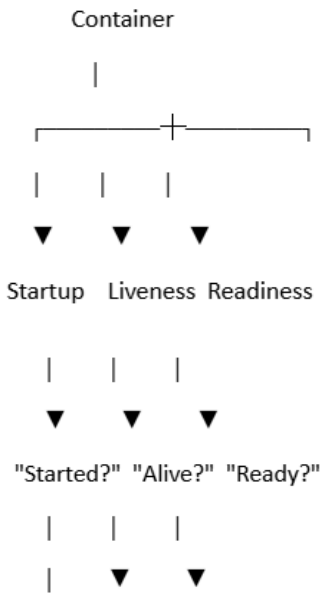
> "Is this container/application still alive enough that restarting it might recover it?"
>
> For example:
>
> Application
>
> |
>
> ├── HTTP traffic → 500 errors
>
> |
>
> └── Liveness endpoint → 200 OK
>
> The liveness probe may still succeed.
>
> Therefore:
>
> Liveness = Should Kubernetes restart this container?
>
> Whereas:
>
> Readiness = Should Kubernetes send traffic to this Pod?
>
> These are different questions.

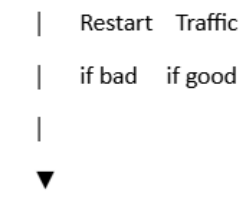
4. Liveness vs Readiness vs Startup

Kubernetes has three important application health mechanisms:

Probe	Main Question	Failure Action
Liveness	Is the application alive?	Restart container
Readiness	Can the application receive traffic?	Remove Pod from Service endpoints
Startup	Has the application finished starting?	Delay liveness/readiness checks

Think of them like this:





Practical 1 — Liveness Probe

We'll create an Nginx pod with an HTTP liveness probe.

Step 1: Create YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: liveness-demo
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
      livenessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 10
        timeoutSeconds: 2
        failureThreshold: 3
```

Save as:

liveness.yaml

Step 2: Create Pod

```
kubectl apply -f liveness.yaml
```

Step 3: Check Pod

kubectl get pod liveness-demo

Expected:

NAME READY STATUS RESTARTS

liveness-demo 1/1 Running 0

Step 4: Check probe configuration

kubectl describe pod liveness-demo

Look for:

Liveness: http-get http://:80/ delay=5s

Practical 2 — Make Liveness Probe Fail

Now let's intentionally break the Nginx health endpoint.

Step 1: Change probe path

Change:

path: /

to:

path: /health

Nginx doesn't have /health, so it returns:

404

Apply:

kubectl apply -f liveness.yaml

Step 2: Watch the Pod

kubectl get pod liveness-demo -w

Eventually you'll see the restart count increase:

NAME READY STATUS RESTARTS

liveness-demo 1/1 Running 1

Check:

kubectl describe pod liveness-demo

You should see events related to the failed liveness probe.

Important concept

Probe fails

↓

Failure threshold reached

↓

Container killed

↓

Container restarted

5. How Kubernetes handles a failed Liveness Probe

Suppose you configure:

Liveness Probe

↓

Every 10 seconds

↓

Application health check

↓

FAIL

One failure does **not necessarily mean immediate restart**.

Kubernetes uses parameters such as:

- periodSeconds
- failureThreshold
- initialDelaySeconds
- timeoutSeconds

For example:

periodSeconds = 10

failureThreshold = 3

Conceptually:

10 sec → FAIL

20 sec → FAIL

30 sec → FAIL

↓

Threshold reached

↓

Restart container

So:

Liveness failure + failure threshold reached → container restart

6. What exactly gets restarted?

A liveness failure generally causes Kubernetes to **restart the affected container**, not necessarily recreate the entire Pod.

For example:

Pod

├— Container A

└— Container B

If Container A's liveness probe fails:

Container A → Restart

Container B → Continues running

The Pod itself may remain the same Pod.

This is different from deleting and recreating a Pod.

7. Liveness Probe Types

Kubernetes supports several mechanisms for liveness checks.

1. HTTP probe

Kubernetes makes an HTTP request to an endpoint.

Conceptually:

Kubernetes

|

| HTTP GET

▼

/health

|

▼

Application

|

```
    |
    └── HTTP response
```

For example:

```
GET /health
```

A successful HTTP response indicates that the probe succeeded.

2. TCP probe

Kubernetes attempts to establish a TCP connection to a specified port.

Conceptually:

```
Kubernetes
    |
    | TCP connection
    ▼
Port 8080
    |
    ▼
Application
```

If Kubernetes can establish the connection, the probe succeeds.

Useful when an application doesn't expose an HTTP health endpoint.

3. Exec probe

Kubernetes executes a command inside the container.

Conceptually:

Kubernetes

|

| execute command

▼

Container

|

└── command

|

├── exit 0 → SUCCESS

└── non-zero → FAILURE

The command's exit status determines whether the probe succeeds.

4. gRPC probe

For applications using gRPC, Kubernetes can use a gRPC health check.

Conceptually:

Kubernetes

|

| gRPC health check

▼

gRPC Application

|

▼

Health status

This is particularly useful for native gRPC services.

8. Probe Result

A probe essentially produces one of these outcomes:

SUCCESS

FAILURE

UNKNOWN

SUCCESS

The health check passed.

Application

↓

Probe succeeds

↓

Continue running

FAILURE

The health check failed.

Application

↓

Probe fails

↓

Failure threshold reached

↓

Container restarted

UNKNOWN

Kubernetes couldn't determine the result properly.

This is different from an application explicitly returning a failed health result.

Practical — TCP Probe

Probes don't have to use HTTP.

Example:

```
livenessProbe:
  tcpSocket:
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10
```

Kubernetes attempts to establish a TCP connection to:

Pod IP:8080

Pod IP:8080

Useful for applications such as:

PostgreSQL

Redis

RabbitMQ

custom TCP applications

Practical — Exec Probe

You can execute a command inside the container.

```
livenessProbe:
  exec:
    command:
      - cat
      - /tmp/healthy
  initialDelaySeconds: 5
```

periodSeconds: 5

Create the file:

kubectl exec liveness-demo -- touch /tmp/healthy

Check:

kubectl exec liveness-demo -- ls -l /tmp/healthy

Remove it:

kubectl exec liveness-demo -- rm /tmp/healthy

Now the probe fails.

9. Important Liveness Probe Configuration Parameters

A liveness probe can have several timing-related parameters.

initialDelaySeconds

Defines how long Kubernetes waits before performing the initial liveness check.

Conceptually:

Container starts

|

| wait

▼

initialDelaySeconds

|

▼

First liveness check

This can be useful for applications that need some time before they can respond.

However, for slow-starting applications, using startupProbe is generally a better design than simply making the liveness delay very large.

periodSeconds

Defines how frequently Kubernetes performs the probe.

For example:

periodSeconds: 10

Conceptually:

0s Probe

10s Probe

20s Probe

30s Probe

40s Probe

---

timeoutSeconds

Defines how long Kubernetes waits for the probe to respond.

For example:

timeoutSeconds: 2

If the probe doesn't complete within the configured timeout, it is considered unsuccessful.

failureThreshold

Defines how many consecutive failures are required before Kubernetes takes the failure action.

For example:

failureThreshold: 3

Conceptually:

Failure #1 → Continue

Failure #2 → Continue

Failure #3 → Restart

This helps avoid restarting an application because of one temporary problem.

successThreshold

Defines how many consecutive successes are required to consider a probe successful.

For liveness probes, Kubernetes uses a success threshold of **1**; successThreshold cannot be increased for liveness.

This is different from readiness probes, where a higher success threshold can be useful.

10. Why shouldn't we make Liveness too aggressive?

Suppose:

periodSeconds = 2

failureThreshold = 1

timeoutSeconds = 1

A temporary network delay or CPU spike could cause:

Probe failure

↓

Immediate restart

Then:

Application starts

↓

Temporary problem

↓

Probe fails

↓

Restart

↓

Application starts again

↓

Probe fails

↓

Restart

This can create a **restart loop**.

Therefore, liveness probes should be designed carefully.

11. Liveness Probe and CrashLoopBackOff

A badly designed liveness probe can contribute to:

CrashLoopBackOff

For example:

Application starts

↓

Liveness probe fails

↓

Container restarted

↓

Application starts

↓

Liveness probe fails

↓

Container restarted

↓

Repeated failures

↓

Back-off

Kubernetes progressively backs off container restarts.

So when you see:

CrashLoopBackOff

one possible cause is an incorrectly configured liveness probe.

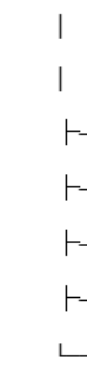
It isn't the only possible cause, though.

12. Liveness Probe and Startup Probe

This is especially important for Java applications.

Imagine:

Spring Boot application



Suppose startup takes 90 seconds.

If liveness starts checking too early:



Then the application may never get enough time to start.

A startupProbe can solve this problem.

Conceptually:

▼

Startup Probe

|

| SUCCESS

▼

Liveness Probe starts becoming relevant

|

▼

Application monitoring

So:

Startup probe protects slow-starting applications from premature liveness restarts.

13. Liveness Probe Should Usually Be Lightweight

A good liveness check should answer a relatively simple question:

"Is the application process still functioning?"

It should generally be:

- Fast
- Lightweight
- Reliable
- Deterministic
- Independent of unnecessary external dependencies

For example, consider:

/health/live

versus:

/health

├── Database

├── Redis

├── Kafka

├── External API

└── 10 other services

If your liveness check depends on every external system, you can accidentally cause unnecessary restarts.

14. Why database checks in Liveness can be dangerous

Suppose:

Application → Database

Database temporarily goes down.

If your liveness endpoint checks the database:

Liveness

↓

Database

↓

Database unavailable

↓

Liveness FAIL

↓

↓

Container restart

But restarting the application doesn't necessarily fix the database.

You could end up with:

Database unavailable

↓

Application restart

↓

Database unavailable

↓

Application restart

Application restart

↓

Application restart

This is undesirable.

Instead, database dependency problems are often more appropriately reflected in **readiness**, depending on the application's architecture.

15. Liveness Probe and Services

Another important distinction:

Liveness Probe

↓

Container health

↓

Restart if unhealthy

Whereas:

Readiness Probe

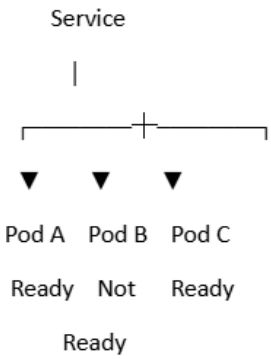
↓

Pod available?

↓

Service routing

For example:



Pod B can remain running but be removed from Service traffic.

That's why:

Readiness failure does not normally restart the container.

While:

Liveness failure can cause a container restart.

16. A Real-World Example

Imagine a Java application running in Kubernetes:

Pod

|

└── Java Application

|

├── JVM running

├── Threads running

└── Application internally deadlocked

The container process is still alive:

Container = Running

But the application isn't functioning properly.

Without liveness:

Kubernetes sees:

Container → Running

Users see:

Application → Broken

With liveness:

Kubernetes

|

▼

Liveness check

|

▼

Application unhealthy

|

▼

Repeated failure

|

▼

Container restarted

|

▼

Application recovers

This is the primary purpose of a liveness probe.

17. Liveness Probe Does NOT Guarantee Zero Downtime

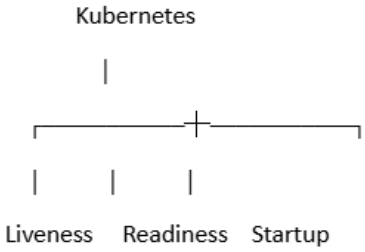
A common misconception is:

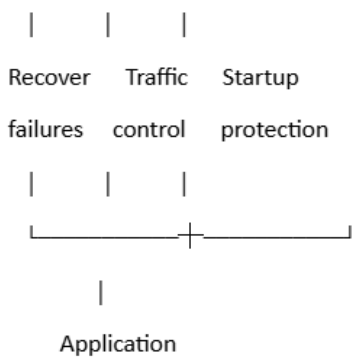
"If I configure liveness, my application will have zero downtime."

Not necessarily.

Liveness primarily helps Kubernetes **recover unhealthy containers**.

For availability, you typically need several things working together:





And at the deployment level:

Deployment

+

Multiple replicas

+

Readiness probe

+

Liveness probe

+

Proper rolling update

can provide much better resilience.

18. Liveness Probe — Interview Definition

For an interview, you can say:

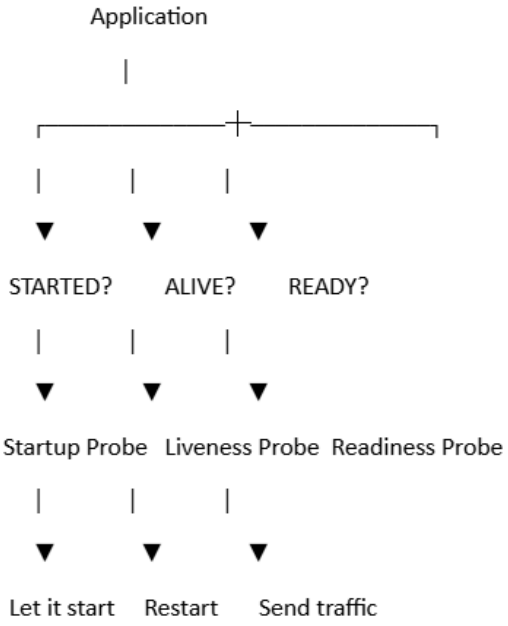
A liveness probe is a Kubernetes health-check mechanism used to determine whether a containerized application is still functioning. Kubernetes periodically executes the configured liveness check, such as HTTP, TCP, gRPC, or an exec command. If the probe fails repeatedly according to the configured failure threshold, Kubernetes restarts the

---

affected container. Liveness probes are primarily intended to recover from application states such as deadlocks, hangs, or unrecoverable internal failures, and should be designed independently from readiness checks.

19. The Most Important Concept to Remember

Think about these three questions:



In one line:

Liveness → "Should I restart this container?"

Readiness → "Should I send traffic to this Pod?"

Startup → "Has this application finished starting?"

That distinction is one of the most frequently tested Kubernetes interview concepts.